AI 냉장고 파먹기 — 앱 설계 문서

항목	내용
작성일	2026-11-03 (10주차)
관련 문서	`ai_prd.md`
기술 스택	Flutter, Provider, Cloud Firestore, Google Cloud Vision API, OpenAI API

1. 아키텍처: MVVM

화면(View)은 상태를 직접 갖지 않고 ViewModel을 구독한다. 데이터 접근은 Repository로 분리하여, 데이터 소스(더미 → Firestore) 교체 시 View/ViewModel을 수정하지 않는다.

```
View (Widget)
  │  context.watch / context.read
  ▼
ViewModel (ChangeNotifier)      ← 화면 상태, 로딩/에러 처리, 비즈니스 로직
  │  호출
  ▼
Repository (abstract class)     ← 데이터 접근 인터페이스
  │  구현체 교체 가능
  ├─ Dummy...Repository         (11주차: 더미 데이터)
  └─ Firestore...Repository     (13주차: 실제 DB)
```

계층	책임	하지 말아야 할 것
View	UI 렌더링, 사용자 입력 전달	비즈니스 로직, API 직접 호출
ViewModel	상태 보관, `notifyListeners()`, Repository 호출	`BuildContext`, 위젯 import
Repository	Firestore/API 통신, 모델 변환	UI 상태 보관
Model	불변 데이터 클래스, `fromMap/toMap`	로직

2. 폴더 구조

```
lib/
├── main.dart                       # 진입점 (Firebase 초기화 예정)
├── app.dart                        # MaterialApp, MultiProvider
├── core/
│   ├── theme/app_theme.dart
│   ├── widgets/                    # 공용 위젯
│   ├── constants/                  # (예정) 문자열, 색상, 컬렉션명
│   └── services/                   # (예정) 알림, OCR, LLM 클라이언트
└── features/
    ├── main/                       # 하단 탭 셸
    │   ├── view/main_screen.dart, widgets/
    │   └── viewmodel/main_viewmodel.dart
    ├── fridge/                     # 내 냉장고 (F2)
    │   ├── model/ingredient.dart
    │   ├── repository/
    │   ├── viewmodel/fridge_viewmodel.dart
    │   └── view/ (+ widgets/)
    ├── scan/                       # 영수증 스캔 (F1)
    │   ├── model/receipt_item.dart
    │   ├── repository/
    │   ├── viewmodel/scan_viewmodel.dart
    │   └── view/ (+ widgets/)
    └── recipe/                     # AI 레시피 (F3)
        ├── model/recipe.dart
        ├── repository/
        ├── viewmodel/recipe_viewmodel.dart
        └── view/ (+ widgets/)
```

규칙

- 기능 폴더끼리 직접 import 하지 않는다. 공유가 필요한 모델(`Ingredient`)은 `fridge/model`을 Repository 인터페이스 경유로만 사용한다.
- 한 파일이 약 150줄을 넘으면 위젯을 `widgets/`로 분리한다.

3. 화면 구성

탭	화면	주요 요소
1. 내 냉장고	`FridgeScreen`	유통기한순 목록, D-Day 색상 배지, 스와이프/체크 소진 처리, 재료 선택 모드
2. 영수증 스캔	`ScanScreen`	카메라 촬영 / 갤러리 선택 버튼
	`ReceiptReviewScreen`	OCR+LLM 결과 목록, 수정/삭제, 저장 버튼
3. AI 레시피	`RecipeScreen`	선택 재료 표시, 생성 버튼, 로딩(Shimmer), 결과(요리명/시간/조리 순서)

화면 흐름

```
[스캔 탭] 촬영/갤러리 → (로딩) → [ReceiptReviewScreen] 수정·삭제 → 저장
                                                                  |
                                                                  ▼
[냉장고 탭] 목록(유통기한순) → 재료 선택 → [레시피 탭] 생성 요청 → (로딩) → 결과
```

4. 데이터 모델

Ingredient (냉장고 식재료)

필드	타입	설명
id	String	Firestore 문서 ID
name	String	식재료명
purchasedAt	DateTime	등록(구매)일
expiryDate	DateTime	유통기한
(getter) daysLeft	int	남은 일수 = `expiryDate - 오늘`

ReceiptItem (스캔 결과, 저장 전 임시 데이터)

필드	타입	설명
name	String	LLM이 추출한 식재료명
suggestedExpiry	DateTime	LLM이 제안한 유통기한 (사용자 수정 가능)

Recipe

필드	타입	설명
title	String	요리명
cookingMinutes	int	소요 시간(분)
steps	List<String>	조리 순서

5. Firestore 구조 (13주차 연동 예정)

회원가입은 MVP 제외이므로 단일 컬렉션으로 시작한다.

```
ingredients/{ingredientId}
  ├─ name: string
  ├─ purchasedAt: timestamp
  └─ expiryDate: timestamp
```

- 목록 조회: `orderBy('expiryDate')` 오름차순 → 유통기한 임박순.
- 소진 처리: 문서 삭제.
- 사용자 구분이 필요해지면 `users/{uid}/ingredients/{id}`로 확장한다. (익명 로그인 도입 시)

6. 핵심 비동기 흐름: 영수증 → 식재료 (F1)

```
이미지 선택 (image_picker)
  → Vision API: 이미지 → 텍스트 (OCR)
  → OpenAI API: 텍스트 → 식재료명 + 기본 유통기한 JSON
  → ScanViewModel: List<ReceiptItem> 상태 갱신
  → 사용자 수정/확인
  → FridgeRepository.addAll() → Firestore 저장
```

`ScanViewModel` 상태: `idle → loading → success | failure`

- `failure`(OCR 인식 불가, JSON 파싱 실패, 네트워크 오류)일 때 크래시 없이 "직접 입력하기"로 유도한다. (PRD 8장)
- 목표 응답 시간: 업로드 ~ 정제 완료 5~7초 이내.

7. 전역 서비스 및 보안

항목	계획
로컬 푸시 알림 (F4)	`flutter_local_notifications`로 유통기한 1~2일 전 예약. 재료 저장/삭제 시 알림 갱신
API 키	`.env`에 저장하고 `.gitignore`에 등록. 저장소에는 `.env.example`만 커밋
키 노출 한계	앱에 포함된 키는 추출될 수 있음. 학습 프로젝트 범위에서는 사용량 제한/무료 크레딧으로 관리

8. 주차별 구현 계획

주차	작업
11	더미 Repository 기반 3개 탭 UI (냉장고 목록, 스캔 화면, 레시피 UI)
13	Vision + OpenAI 연동, Firestore Repository 구현, 레시피 생성
14	푸시 알림, OCR 실패 예외 처리, 통합 테스트

9. 현재 구현 상태 (10주차 종료 시점)

- [x] Flutter 프로젝트 생성, provider 적용
- [x] `main.dart`, `app.dart`, 테마
- [x] 하단 네비게이션 `MainScreen` + `MainViewModel`
- [x] 3개 탭 더미 화면
- [ ] 와이어프레임 (Figma)